

Iterative Methods

Synopsis

We present several methods for solving linear inverse problems that may be far too large for the methods previously discussed to be practical. These methods are iterative, in that a sequence of trial solutions is generated that converges to a final solution. Kaczmarz's algorithm and the related ART and SIRT methods form one class, while methods based on conjugate gradients form a second class. When the method of conjugate gradients is applied to the normal equations, the resulting conjugate gradient least squares (CGLS) method regularizes the solution to the inverse problem. Methods for estimating model resolution for very large model spaces are described. Illustrative examples involving tomography and image deblurring are given.

6.1. INTRODUCTION

Singular value decomposition (SVD)-based pseudoinverse and Tikhonov regularization solutions become impractical when we consider larger problems in which $\mathbf{G}$ has, for example, tens of thousands of rows and columns. Storing all of the elements in such a large $\mathbf{G}$ matrix can require a great deal of computer memory. If many of the elements in the $\mathbf{G}$ matrix are zero, as for example in ray-theory tomography, then $\mathbf{G}$ is a **sparse matrix**, and we can obviate this problem by storing only the nonzero elements of $\mathbf{G}$ and their locations. The **density** of $\mathbf{G}$ is the percentage of nonzero elements in the matrix. MATLAB can store sparse matrices and most built-in MATLAB functions operate efficiently on sparse matrices. **Dense matrices** contain enough nonzero elements that sparse storage schemes are not efficient.

Methods for the solution of linear systems of equations based on matrix factorizations such as the Cholesky factorization, QR factorization, or SVD do not tend to work well with sparse matrices. The problem is that the matrices that occur in the factorization of $\mathbf{G}$ are often more dense than $\mathbf{G}$ itself. In particular, the $\mathbf{U}$ and $\mathbf{V}$ matrices in the SVD and the $\mathbf{Q}$ matrix in the QR factorization are orthogonal matrices, which typically make these matrices dense.

The iterative methods discussed in this chapter do not require the storage of large dense matrices. Instead, they work by generating a sequence of models $\mathbf{m}$ that converge to an optimal solution. Steps in these algorithms typically involve multiplying $\mathbf{G}$ and $\mathbf{G}^T$ times vectors, which can be done without additional storage. Because iterative methods can also take advantage of the sparsity commonly found in the $\mathbf{G}$ matrix, they are often used for very large problems.

For example, consider a two-dimensional tomography problem where the model is of size 256 by 256 (65,536 model elements) and there are 100,000 ray paths. Each ray path will pass through less than 1% of the model cells, so the vast majority of the elements (e.g., more than 99%) in $\mathbf{G}$ will be zero. If we stored $\mathbf{G}$ as a full matrix, it would require about 50 gigabytes (GB) of storage. Furthermore, if we wished to apply the SVD, the $\mathbf{U}$ matrix would require 80 GB of storage, and the $\mathbf{V}$ matrix would require about 35 GB. Because of the extreme sparsity of $\mathbf{G}$, such a matrix can be stored in less than 1 GB using sparse storage.

At the time this book was written, computers with several gigabytes of main memory were common, while only computers classified as supercomputers would have hundreds or thousands of gigabytes of main memory. The point at which it becomes necessary to use sparse matrix storage depends on the computer that we are using. Although the memory capacity of computers has been increasing steadily for many years, it is safe to say that there will always be problems for which sparse matrix storage will be required. Furthermore, programs that use sparse matrix storage often run much faster than programs that do not take advantage of sparsity. Thus it is a good idea to consider using sparse storage even in cases where sufficient memory is available to store the full matrix.

6.2. ITERATIVE METHODS FOR TOMOGRAPHY PROBLEMS

We will concentrate in this section on Kaczmarz's algorithm and its ART and SIRT variants. These algorithms were originally developed for tomographic applications and are particularly effective for such problems.

Kaczmarz's algorithm is an easy-to-implement algorithm for solving a linear system of equations $\mathbf{G}\mathbf{m} = \mathbf{d}$. To understand the algorithm, note that each of the m equations $\mathbf{G}_i \cdot \mathbf{m} = d_i$ defines an n -dimensional hyperplane in R^m . Kaczmarz's algorithm starts with an initial solution $\mathbf{m}^{(0)}$, and then moves to a solution $\mathbf{m}^{(1)}$ by projecting the initial solution onto the hyperplane defined by the first row in $\mathbf{G}$. Next $\mathbf{m}^{(1)}$ is similarly projected onto the hyperplane defined by the second row in $\mathbf{G}$, and so forth. The process is repeated until the solution has been projected onto all m hyperplanes defined by the system of equations. At that point, a new cycle of projections begins. These cycles are repeated until the solution has converged sufficiently. [Figure 6.1](#) shows an example

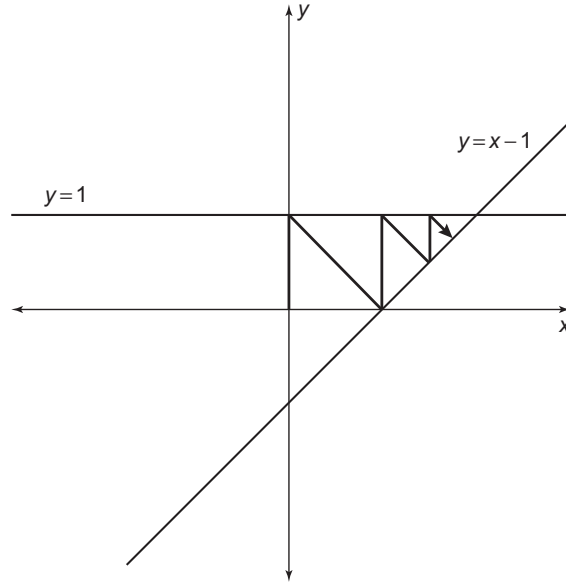


Figure 6.1 Kaczmarz's algorithm on a system of two equations.

in which Kaczmarz's algorithm is used to solve the system of equations

$$\begin{aligned} y &= 1 \\ -x + y &= -1. \end{aligned} \tag{6.1}$$

To implement the algorithm, we need a formula to compute the projection of a vector onto the hyperplane defined by equation i . Consider the hyperplane defined by $\mathbf{G}_{i+1, \cdot} \mathbf{m} = d_{i+1}$. Because the vector $\mathbf{G}_{i+1, \cdot}^T$ is perpendicular to this hyperplane, the update to $\mathbf{m}^{(i)}$ from the constraint due to row $i+1$ of $\mathbf{G}$ will be proportional to $\mathbf{G}_{i+1, \cdot}^T$.

$$\mathbf{m}^{(i+1)} = \mathbf{m}^{(i)} + \beta \mathbf{G}_{i+1, \cdot}^T. \tag{6.2}$$

Using the fact that $\mathbf{G}_{i+1, \cdot} \mathbf{m}^{(i+1)} = d_{i+1}$ to solve for β , we obtain

$$\mathbf{G}_{i+1, \cdot} (\mathbf{m}^{(i)} + \beta \mathbf{G}_{i+1, \cdot}^T) = d_{i+1} \tag{6.3}$$

$$\mathbf{G}_{i+1, \cdot} \mathbf{m}^{(i)} - d_{i+1} = -\beta \mathbf{G}_{i+1, \cdot} \mathbf{G}_{i+1, \cdot}^T. \tag{6.4}$$

$$\beta = -\frac{\mathbf{G}_{i+1, \cdot} \mathbf{m}^{(i)} - d_{i+1}}{\mathbf{G}_{i+1, \cdot} \mathbf{G}_{i+1, \cdot}^T}. \tag{6.5}$$

Thus the update formula is

$$\mathbf{m}^{(i+1)} = \mathbf{m}^{(i)} - \frac{\mathbf{G}_{i+1,\cdot} \mathbf{m}^{(i)} - d_{i+1}}{\mathbf{G}_{i+1,\cdot} \mathbf{G}_{i+1,\cdot}^T} \mathbf{G}_{i+1,\cdot}^T \quad (6.6)$$

It can be shown that if the system of equations $\mathbf{G}\mathbf{m} = \mathbf{d}$ has a unique solution, then Kaczmarz's algorithm will converge to this solution. If the system of equations has many solutions, then the algorithm will converge to the solution that is closest to the point $\mathbf{m}^{(0)}$. In particular, if we start with $\mathbf{m}^{(0)} = \mathbf{0}$, we will obtain a minimum length solution. If there is no exact solution to the system of equations, then the algorithm will fail to converge, but will typically bounce around near an approximate solution.

A second important question is how quickly Kaczmarz's algorithm will converge to a solution. If the hyperplanes described by the system of equations are nearly orthogonal, then the algorithm will converge very quickly. However, if two or more hyperplanes are nearly parallel to each other, convergence can be extremely slow. Figure 6.2 shows a typical situation in which the algorithm zigzags back and forth without making much progress towards a solution. As the two lines become more nearly parallel, the problem becomes worse. This problem can be alleviated by picking an ordering of the equations such that adjacent equations describe hyperplanes that are nearly orthogonal to each other. In the context of tomography, this can be done by ordering the equations so that successive equations do not share common model cells.

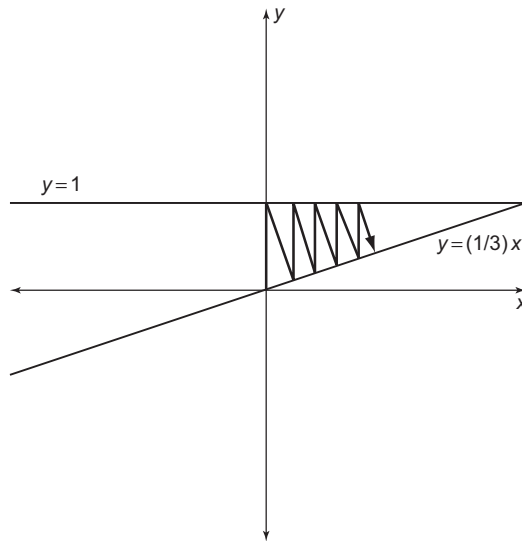


Figure 6.2 Slow convergence occurs when hyperplanes are nearly parallel.



Example 6.1

Consider a tomographic reconstruction problem with the same geometry used in Exercise 4.4, in which the slowness structure is parameterized in homogeneous blocks of size l by l . The true model is shown in Figure 6.3. Synthetic data were generated, with normally distributed random noise added. The random noise had a standard deviation of 0.01. Figure 6.4 shows the TSVD solution. The two anomalies are apparent, but it is not possible to distinguish the small hole within the larger of the two.

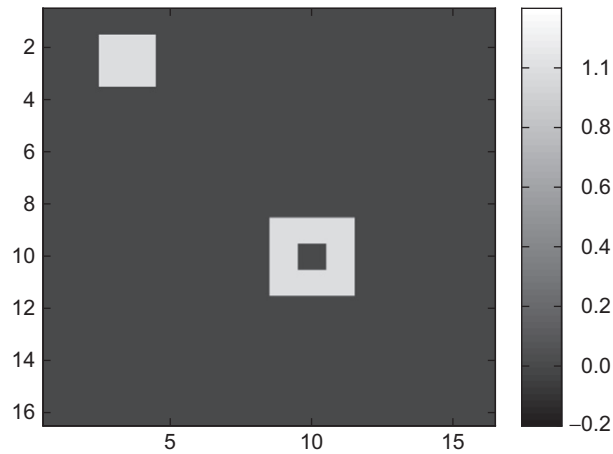


Figure 6.3 True model.

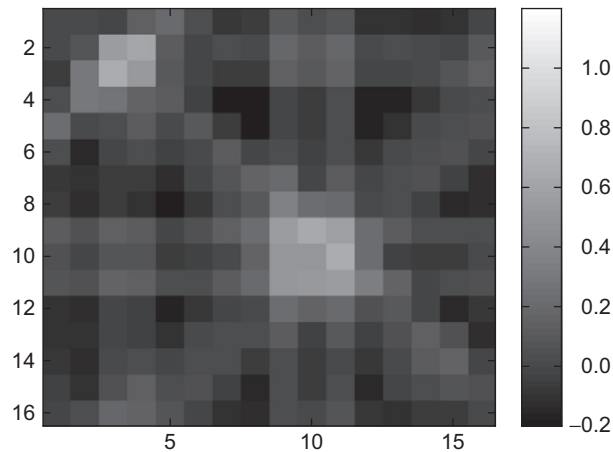


Figure 6.4 Truncated SVD solution.

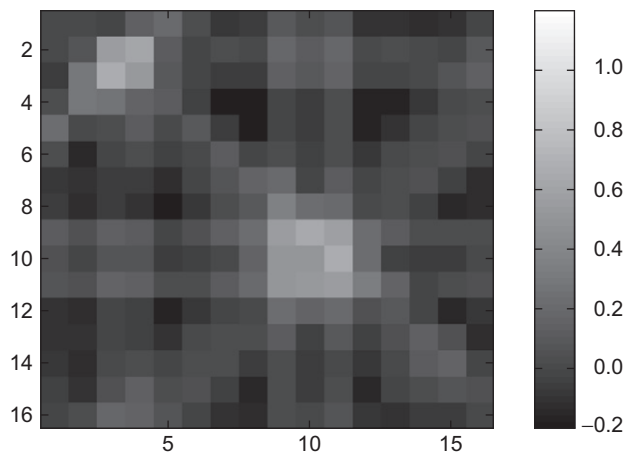


Figure 6.5 Kaczmarz's algorithm solution.

Figure 6.5 shows the solution obtained after 200 iterations of Kaczmarz's algorithm. This solution is extremely similar to the TSVD solution, and both solutions are about the same distance from the true model.

The algebraic reconstruction technique (ART) is a version of Kaczmarz's algorithm that has been modified especially for the tomographic reconstruction problem. In (6.6), the updates to the solution always consist of adding a multiple of a row of $\mathbf{G}$ to the current solution. The numerator in the fraction is the difference between the right-hand side of equation $i + 1$ and the value of the $i + 1$ component of $\mathbf{G}\mathbf{m}$. The denominator in (6.6) is simply the square of the norm of $\mathbf{G}_{i+1,\cdot}$. Effectively, Kaczmarz's algorithm is determining the error in equation $i + 1$, and then adjusting the solution by spreading the required correction over the elements of $\mathbf{m}$ that appear in equation $i + 1$.

Algorithm 6.1 Kaczmarz's Algorithm

Given a system of equations $\mathbf{G}\mathbf{m} = \mathbf{d}$.

1. Let $\mathbf{m}^{(0)} = \mathbf{0}$.
2. For $i = 0, 1, \dots, m - 1$, let

$$\mathbf{m}^{(i+1)} = \mathbf{m}^{(i)} - \frac{\mathbf{G}_{i+1,\cdot}\mathbf{m}^{(i)} - d_{i+1}}{\mathbf{G}_{i+1,\cdot}\mathbf{G}_{i+1,\cdot}^T} \mathbf{G}_{i+1,\cdot}^T \quad (6.7)$$

3. If the solution has not yet converged, return to step 2.
-

A crude approximation to the Kaczmarz update, used in ART, is to replace all of the nonzero elements in row $i + 1$ of $\mathbf{G}$ with ones. We define

$$q_{i+1} = \sum_{\text{cell } j \text{ in ray path } i+1} \mathbf{m}_j l \quad (6.8)$$

as an approximation to the travel time along ray path $i + 1$, where l is the cell dimension. The difference between q_{i+1} and d_{i+1} is roughly the residual for the predicted travel time of ray $i + 1$.

Examining (6.6) for the ART-modified $\mathbf{G}$, we see that ART simply takes the total travel time misfit for ray $i + 1$ and divides it by l times the number of cells in the ray path, N_{i+1} . This correction factor is then multiplied by a vector that has ones in cells along ray path $i + 1$. This procedure has the effect of smearing the needed correction in travel time equally over all of the cells in the ray path.

The ART approximate update formula can thus be written as

$$m_j^{(i+1)} = \begin{cases} m_j^{(i)} - \frac{q_{i+1} - d_{i+1}}{lN_{i+1}} & \text{cell } j \text{ in ray path } i + 1 \\ m_j^{(i)} & \text{cell } j \text{ not in ray path } i + 1. \end{cases} \quad (6.9)$$

The approximation can be improved by taking into account that the ray path lengths actually will vary from cell to cell. If L_{i+1} is the length of ray path $i + 1$, the corresponding improved update formula from (6.6) for the tomography problem is

$$m_j^{(i+1)} = \begin{cases} m_j^{(i)} + \frac{d_{i+1}}{L_{i+1}} - \frac{q_{i+1}}{lN_{i+1}} & \text{cell } j \text{ in ray path } i + 1 \\ m_j^{(i)} & \text{cell } j \text{ not in ray path } i + 1. \end{cases} \quad (6.10)$$

The main advantage of ART is that it saves storage. We need only store information about which rays pass through which cells, and we do not need to record the length of each ray in each cell. A second advantage of the method is that it reduces the number of floating-point multiplications compared to that required by Kaczmarz's algorithm. Although in current computers floating-point multiplications and additions require roughly the same amount of time, during the 1970s, when ART was first developed, multiplication was slower than addition.

Algorithm 6.2 ART

Given a system of equations $\mathbf{Gm} = \mathbf{d}$ arising from a tomography problem:

1. Let $\mathbf{m}^{(0)} = \mathbf{0}$.
 2. For $i = 0, 1, \dots, m$, let N_i be the number of cells traversed by ray path i .
 3. For $i = 0, 1, \dots, m$, let L_i be the length of ray path i .
-

Algorithm 6.2 (Continued)

4. For $i = 0, 1, \dots, m-1, j = 1, 2, \dots, n$, let

$$m_j^{(i+1)} = \begin{cases} m_j^{(i)} + \frac{d_{i+1}}{L_{i+1}} - \frac{q_{i+1}}{IN_{i+1}} & \text{cell } j \text{ in ray path } i+1 \\ m_j^{(i)} & \text{cell } j \text{ not in ray path } i+1. \end{cases} \quad (6.11)$$

5. If the solution has not yet converged, let $\mathbf{m}^{(0)} = \mathbf{m}^{(m)}$ and return to step 4. Otherwise, return the solution $\mathbf{m} = \mathbf{m}^{(m)}$.

One problem with ART is that the resulting tomographic images tend to be noisier than images produced by Kaczmarz's algorithm (6.7). The simultaneous iterative reconstruction technique (SIRT) is a variation on ART that gives slightly better images in practice, at the expense of a slightly slower algorithm. In the SIRT algorithm, all (up to m nonzero) updates using (6.10) are computed for each cell j of the model, for each ray that passes through cell j . The set of updates for cell j are then averaged before updating the corresponding model element m_j .

Algorithm 6.3 SIRT

Given a system of equations $\mathbf{Gm} = \mathbf{d}$ arising from a tomography problem:

1. Let $\mathbf{m}^{(0)} = \mathbf{0}$.
2. For $j = 0, 1, \dots, n$, let K_j be the number of ray paths that pass through cell j .
3. For $i = 0, 1, \dots, m$, let L_i be the length of ray path i .
4. For $i = 0, 1, \dots, m$, let N_i be the number of cells traversed by ray path i .
5. Let $\Delta \mathbf{m}^{(i+1)} = \mathbf{0}$.
6. For $i = 0, 1, \dots, m-1, j = 1, 2, \dots, n$, let

$$\Delta m_j^{(i+1)} = \Delta m_j^{(i+1)} + \begin{cases} \frac{d_{i+1}}{L_{i+1}} - \frac{q_{i+1}}{IN_{i+1}} & \text{cell } j \text{ in ray path } i+1 \\ 0 & \text{cell } j \text{ not in ray path } i+1. \end{cases} \quad (6.12)$$

7. For $j = 1, 2, \dots, n$, let

$$m_j^{(i+1)} = m_j^{(i+1)} + \frac{\Delta m_j^{(i+1)}}{K_j}. \quad (6.13)$$

8. If the solution has not yet converged, return to step 5. Otherwise, return the current solution.

**Example 6.2**

Returning to our earlier tomography example, [Figure 6.6](#) shows the ART solution obtained after 200 iterations. Again, the solution is very similar to the TSVD solution.

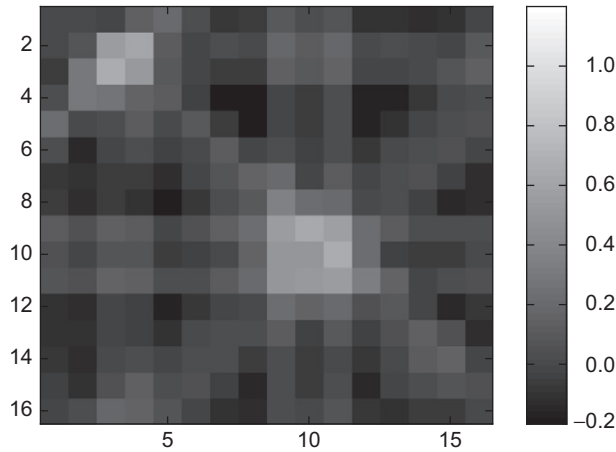


Figure 6.6 ART solution.

[Figure 6.7](#) shows the SIRT solution for our example tomography problem after 200 iterations. This solution is similar to the TSVD, Kaczmarz's, and ART solutions.

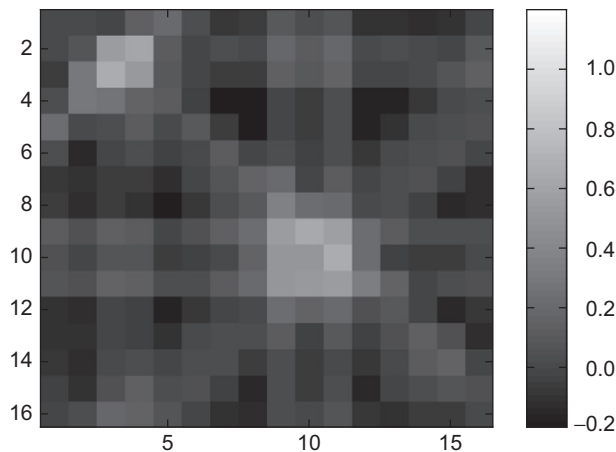


Figure 6.7 SIRT solution.

6.3. THE CONJUGATE GRADIENT METHOD

We next consider the **conjugate gradient** (CG) method for solving a symmetric and positive definite system of equations $\mathbf{Ax} = \mathbf{b}$. We will later apply the CG method to solving the normal equations for $\mathbf{Gm} = \mathbf{d}$. Consider the quadratic optimization problem,

$$\min \phi(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x}, \quad (6.14)$$

where $\mathbf{A}$ is an $n \times n$ symmetric and positive definite matrix. We require $\mathbf{A}$ be positive definite so that the function $\phi(\mathbf{x})$ will be convex and have a unique minimum. We can calculate $\nabla \phi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$ and set it equal to zero to find the minimum. The minimum occurs at a point $\mathbf{x}$ that satisfies the equation

$$\mathbf{Ax} - \mathbf{b} = \mathbf{0} \quad (6.15)$$

or

$$\mathbf{Ax} = \mathbf{b}. \quad (6.16)$$

Thus solving the system of equations $\mathbf{Ax} = \mathbf{b}$ is equivalent to minimizing $\phi(\mathbf{x})$.

The conjugate gradient method approaches the problem of minimizing $\phi(\mathbf{x})$ by constructing a basis for R^n in which the minimization problem is extremely simple. The basis vectors $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{n-1}$ are selected so that

$$\mathbf{p}_i^T \mathbf{Ap}_j = 0 \quad \text{when } i \neq j. \quad (6.17)$$

A collection of vectors with this property is said to be **mutually conjugate** with respect to $\mathbf{A}$. We express $\mathbf{x}$ in terms of these basis vectors as

$$\mathbf{x} = \sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i \quad (6.18)$$

so that

$$\phi(\boldsymbol{\alpha}) = \frac{1}{2} \left(\sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i \right)^T \mathbf{A} \left(\sum_{j=0}^{n-1} \alpha_j \mathbf{p}_j \right) - \mathbf{b}^T \left(\sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i \right). \quad (6.19)$$

The product $\mathbf{x}^T \mathbf{Ax}$ can be written as a double sum, so the equation becomes

$$\phi(\boldsymbol{\alpha}) = \frac{1}{2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \alpha_i \alpha_j \mathbf{p}_i^T \mathbf{Ap}_j - \mathbf{b}^T \left(\sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i \right). \quad (6.20)$$

Since the vectors are mutually conjugate with respect to $\mathbf{A}$, this simplifies to

$$\phi(\boldsymbol{\alpha}) = \frac{1}{2} \sum_{i=0}^{n-1} \alpha_i^2 \mathbf{p}_i^T \mathbf{A} \mathbf{p}_i - \mathbf{b}^T \left(\sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i \right) \quad (6.21)$$

or

$$\phi(\boldsymbol{\alpha}) = \frac{1}{2} \sum_{i=0}^{n-1} (\alpha_i^2 \mathbf{p}_i^T \mathbf{A} \mathbf{p}_i - 2\alpha_i \mathbf{b}^T \mathbf{p}_i). \quad (6.22)$$

Equation (6.22) shows that $\phi(\boldsymbol{\alpha})$ consists of n terms, each of which is independent of the other terms. Thus we can minimize $\phi(\boldsymbol{\alpha})$ by selecting each α_i to minimize the i th term,

$$\alpha_i^2 \mathbf{p}_i^T \mathbf{A} \mathbf{p}_i - 2\alpha_i \mathbf{b}^T \mathbf{p}_i. \quad (6.23)$$

Differentiating with respect to α_i and setting the derivative equal to zero, we find that the optimal value for α_i is

$$\alpha_i = \frac{\mathbf{b}^T \mathbf{p}_i}{\mathbf{p}_i^T \mathbf{A} \mathbf{p}_i}. \quad (6.24)$$

This shows that if we have a basis of vectors that are mutually conjugate with respect to $\mathbf{A}$, then minimizing $\phi(\mathbf{x})$ is very easy. We have not yet shown how to construct the mutually conjugate basis vectors.

Our algorithm will actually construct a sequence of solution vectors $\mathbf{x}_i$, residual vectors $\mathbf{r}_i = \mathbf{b} - \mathbf{A}\mathbf{x}_i$, and basis vectors $\mathbf{p}_i$. The algorithm begins with $\mathbf{x}_0 = \mathbf{0}$, $\mathbf{r}_0 = \mathbf{b}$, $\mathbf{p}_0 = \mathbf{r}_0$, and $\alpha_0 = (\mathbf{r}_0^T \mathbf{r}_0) / (\mathbf{p}_0^T \mathbf{A} \mathbf{p}_0)$.

Suppose that at the start of iteration k of the algorithm, we have constructed $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_k, \mathbf{r}_0, \mathbf{r}_1, \dots, \mathbf{r}_k, \mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_k$ and $\alpha_0, \alpha_1, \dots, \alpha_k$. We assume that the first $k+1$ basis vectors $\mathbf{p}_i$ are mutually conjugate with respect to $\mathbf{A}$, the first $k+1$ residual vectors $\mathbf{r}_i$ are mutually orthogonal, and that $\mathbf{r}_i^T \mathbf{p}_j = 0$ when $i \neq j$.

We let

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k. \quad (6.25)$$

This effectively adds one more term of (6.18) into the solution. Next, we let

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k. \quad (6.26)$$

This correctly updates the residual, because

$$\mathbf{r}_{k+1} = \mathbf{b} - \mathbf{A}\mathbf{x}_{k+1} \quad (6.27)$$

$$= \mathbf{b} - \mathbf{A}(\mathbf{x}_k + \alpha_k \mathbf{p}_k) \quad (6.28)$$

$$= (\mathbf{b} - \mathbf{A}\mathbf{x}_k) - \alpha_k \mathbf{A}\mathbf{p}_k \quad (6.29)$$

$$= \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k. \quad (6.30)$$

We let

$$\beta_{k+1} = \frac{\|\mathbf{r}_{k+1}\|_2^2}{\|\mathbf{r}_k\|_2^2} = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k} \quad (6.31)$$

and

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_{k+1} \mathbf{p}_k. \quad (6.32)$$

In the following calculations, it will be useful to know that $\mathbf{b}^T \mathbf{p}_k = \mathbf{r}_k^T \mathbf{r}_k$. This is shown by

$$\mathbf{b}^T \mathbf{p}_k = (\mathbf{r}_k + \mathbf{A}\mathbf{x}_k)^T \mathbf{p}_k \quad (6.33)$$

$$= \mathbf{r}_k^T \mathbf{p}_k + \mathbf{x}_k^T \mathbf{A}\mathbf{p}_k \quad (6.34)$$

$$= \mathbf{r}_k^T \mathbf{p}_k + \mathbf{p}_k^T \mathbf{A}\mathbf{x}_k \quad (6.35)$$

$$= \mathbf{r}_k^T (\mathbf{r}_k + \beta_k \mathbf{p}_{k-1}) + \mathbf{p}_k^T \mathbf{A}\mathbf{x}_k \quad (6.36)$$

$$= \mathbf{r}_k^T \mathbf{r}_k + \beta_k \mathbf{r}_k^T \mathbf{p}_{k-1} + \mathbf{p}_k^T \mathbf{A}(\alpha_0 \mathbf{p}_0 + \cdots + \alpha_{k-1} \mathbf{p}_{k-1}) \quad (6.37)$$

$$= \mathbf{r}_k^T \mathbf{r}_k + 0 + 0 \quad (6.38)$$

$$= \mathbf{r}_k^T \mathbf{r}_k. \quad (6.39)$$

We will now show that $\mathbf{r}_{k+1}$ is orthogonal to $\mathbf{r}_i$ for $i \leq k$. For every $i < k$,

$$\mathbf{r}_{k+1}^T \mathbf{r}_i = (\mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k)^T \mathbf{r}_i \quad (6.40)$$

$$= \mathbf{r}_k^T \mathbf{r}_i - \alpha_k \mathbf{p}_k^T \mathbf{A}\mathbf{r}_i \quad (6.41)$$

$$= \mathbf{r}_k^T \mathbf{r}_i - \alpha_k \mathbf{r}_i^T \mathbf{A}\mathbf{p}_k. \quad (6.42)$$

Since $\mathbf{r}_k$ is orthogonal to all of the earlier $\mathbf{r}_i$ vectors,

$$\mathbf{r}_{k+1}^T \mathbf{r}_i = 0 - \alpha_k \mathbf{p}_k^T \mathbf{A}\mathbf{r}_i. \quad (6.43)$$

Also, since $\mathbf{p}_i = \mathbf{r}_i + \beta_i \mathbf{p}_{i-1}$,

$$\mathbf{r}_{k+1}^T \mathbf{r}_i = 0 - \alpha_k (\mathbf{p}_i - \beta_i \mathbf{p}_{i-1})^T \mathbf{A}\mathbf{p}_k. \quad (6.44)$$

Both $\mathbf{p}_i$ and $\mathbf{p}_{i-1}$ are conjugate with $\mathbf{p}_k$. Thus

$$\mathbf{r}_{k+1}^T \mathbf{r}_i = 0. \quad (6.45)$$

We also have to show that $\mathbf{r}_{k+1}^T \mathbf{r}_k = 0$:

$$\mathbf{r}_{k+1}^T \mathbf{r}_k = (\mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k)^T \mathbf{r}_k \quad (6.46)$$

$$= \mathbf{r}_k^T \mathbf{r}_k - \alpha_k \mathbf{p}_k^T \mathbf{A} \mathbf{r}_k \quad (6.47)$$

$$= \mathbf{r}_k^T \mathbf{r}_k - \alpha_k \mathbf{p}_k^T \mathbf{A} (\mathbf{p}_k - \beta_k \mathbf{p}_{k-1}) \quad (6.48)$$

$$= \mathbf{r}_k^T \mathbf{r}_k - \alpha_k (\mathbf{p}_k - \beta_k \mathbf{p}_{k-1})^T \mathbf{A} \mathbf{p}_k \quad (6.49)$$

$$= \mathbf{r}_k^T \mathbf{r}_k - \alpha_k \mathbf{p}_k^T \mathbf{A} \mathbf{p}_k + \alpha_k \beta_k \mathbf{p}_{k-1}^T \mathbf{A} \mathbf{p}_k \quad (6.50)$$

$$= \mathbf{r}_k^T \mathbf{r}_k - \mathbf{r}_k^T \mathbf{r}_k + \alpha_k \beta_k 0 \quad (6.51)$$

$$= 0. \quad (6.52)$$

Next, we will show that $\mathbf{r}_{k+1}$ is orthogonal to $\mathbf{p}_i$ for $i \leq k$.

$$\mathbf{r}_{k+1}^T \mathbf{p}_i = \mathbf{r}_{k+1}^T (\mathbf{r}_i + \beta_i \mathbf{p}_{i-1}) \quad (6.53)$$

$$= \mathbf{r}_{k+1}^T \mathbf{r}_i + \beta_i \mathbf{r}_{k+1}^T \mathbf{p}_{i-1} \quad (6.54)$$

$$= 0 + \beta_i \mathbf{r}_{k+1}^T \mathbf{p}_{i-1} \quad (6.55)$$

$$= \beta_i (\mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k)^T \mathbf{p}_{i-1} \quad (6.56)$$

$$= \beta_i (\mathbf{r}_k^T \mathbf{p}_{i-1} - \alpha_k \mathbf{p}_{i-1}^T \mathbf{A} \mathbf{p}_k) \quad (6.57)$$

$$= \beta_i (0 - 0) = 0. \quad (6.58)$$

Finally, we need to show that $\mathbf{p}_{k+1}^T \mathbf{A} \mathbf{p}_i = 0$ for $i \leq k$. For $i < k$,

$$\mathbf{p}_{k+1}^T \mathbf{A} \mathbf{p}_i = (\mathbf{r}_{k+1} + \beta_{k+1} \mathbf{p}_k)^T \mathbf{A} \mathbf{p}_i \quad (6.59)$$

$$= \mathbf{r}_{k+1}^T \mathbf{A} \mathbf{p}_i + \beta_{k+1} \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i \quad (6.60)$$

$$= \mathbf{r}_{k+1}^T \mathbf{A} \mathbf{p}_i + 0 \quad (6.61)$$

$$= \mathbf{r}_{k+1}^T \left(\frac{1}{\alpha_i} (\mathbf{r}_i - \mathbf{r}_{i+1}) \right) \quad (6.62)$$

$$= \frac{1}{\alpha_i} (\mathbf{r}_{k+1}^T \mathbf{r}_i - \mathbf{r}_{k+1}^T \mathbf{r}_{i+1}) \quad (6.63)$$

$$= 0. \quad (6.64)$$

For $i = k$,

$$\mathbf{p}_{k+1}^T \mathbf{A} \mathbf{p}_k = (\mathbf{r}_{k+1} + \beta_{k+1} \mathbf{p}_k)^T \left(\frac{1}{\alpha_k} (\mathbf{r}_k - \mathbf{r}_{k+1}) \right) \quad (6.65)$$

$$= \frac{1}{\alpha_k} \left(\beta_{k+1} (\mathbf{r}_k + \beta_k \mathbf{p}_{k-1})^T \mathbf{r}_k - \mathbf{r}_{k+1}^T \mathbf{r}_{k+1} \right) \quad (6.66)$$

$$= \frac{1}{\alpha_k} (\beta_{k+1} \mathbf{r}_k^T \mathbf{r}_k + \beta_{k+1} \beta_k \mathbf{p}_{k-1}^T \mathbf{r}_k - \mathbf{r}_{k+1}^T \mathbf{r}_{k+1}) \quad (6.67)$$

$$= \frac{1}{\alpha_k} (\mathbf{r}_{k+1}^T \mathbf{r}_{k+1} + \beta_{k+1} \beta_k 0 - \mathbf{r}_{k+1}^T \mathbf{r}_{k+1}) \quad (6.68)$$

$$= 0. \quad (6.69)$$

We have now shown that the algorithm generates a sequence of mutually conjugate basis vectors. In theory, the algorithm will find an exact solution to the system of equations in n iterations. In practice, due to round-off errors in the computation, the exact solution may not be obtained in n iterations. In practical implementations of the algorithm, we iterate until the residual is smaller than some tolerance that we specify. The algorithm can be summarized as follows.

Algorithm 6.4 Conjugate Gradient Method

Given a positive definite and symmetric system of equations $\mathbf{A}\mathbf{x} = \mathbf{b}$, and an initial solution $\mathbf{x}_0$, let $\beta_{-1} = 0$, $\mathbf{p}_{-1} = \mathbf{0}$, $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, and $k = 0$.

1. Let $\mathbf{p}_k = -\mathbf{r}_k + \beta_{k-1} \mathbf{p}_{k-1}$.
 2. Let $\alpha_k = \frac{\|\mathbf{r}_k\|_2^2}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$.
 3. Let $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$.
 4. Let $\mathbf{r}_{k+1} = \mathbf{r}_k + \alpha_k \mathbf{A} \mathbf{p}_k$.
 5. Let $\beta_k = \frac{\|\mathbf{r}_{k+1}\|_2^2}{\|\mathbf{r}_k\|_2^2}$.
 6. Let $k = k + 1$.
 7. Repeat the previous steps until convergence.
-

A major advantage of the CG method is that it requires storage only for the vectors $\mathbf{x}_k$, $\mathbf{p}_k$, $\mathbf{r}_k$, and the symmetric matrix $\mathbf{A}$. If $\mathbf{A}$ is large and sparse, then sparse matrix techniques can be used to store $\mathbf{A}$ more efficiently. Unlike factorization methods such as QR, SVD, or Cholesky factorization, there will be no fill-in of the zero elements in $\mathbf{A}$ at any stage in the solution process. Thus it is possible to solve extremely large systems using CG in cases where direct factorization would require far too much storage. In fact, the only way in which the algorithm uses $\mathbf{A}$ is in one multiplication of $\mathbf{A} \mathbf{p}_k$ for each

iteration. In some applications of the CG method, it is possible to perform these matrix vector multiplications without explicitly constructing $\mathbf{A}$.

6.4. THE CGLS METHOD

The CG method by itself can only be applied to positive definite systems of equations, and is thus not directly applicable to general least squares problems. In the **conjugate gradient least squares** (CGLS) method, we solve a least squares problem,

$$\min \|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2, \quad (6.70)$$

by applying CG to the normal equations,

$$\mathbf{G}^T \mathbf{G} \mathbf{m} = \mathbf{G}^T \mathbf{d}. \quad (6.71)$$

In implementing this algorithm it is important to avoid round-off errors. One important source of error is the evaluation of the residual, $\mathbf{G}^T \mathbf{G} \mathbf{m} - \mathbf{G}^T \mathbf{d}$. It turns out that this calculation is more accurate when we factor out $\mathbf{G}^T$ and compute $\mathbf{G}^T (\mathbf{G} \mathbf{m} - \mathbf{d})$. We will use the notation $\mathbf{s}_k = \mathbf{G} \mathbf{m}_k - \mathbf{d}$, and $\mathbf{r}_k = \mathbf{G}^T \mathbf{s}_k$. Note that we can compute $\mathbf{s}_{k+1}$ recursively from $\mathbf{s}_k$ as follows

$$\mathbf{s}_{k+1} = \mathbf{G} \mathbf{m}_{k+1} - \mathbf{d} \quad (6.72)$$

$$= \mathbf{G}(\mathbf{m}_k + \alpha_k \mathbf{p}_k) - \mathbf{d} \quad (6.73)$$

$$= (\mathbf{G} \mathbf{m}_k - \mathbf{d}) + \alpha_k \mathbf{G} \mathbf{p}_k \quad (6.74)$$

$$= \mathbf{s}_k + \alpha_k \mathbf{G} \mathbf{p}_k. \quad (6.75)$$

With this trick, we can now state the CGLS algorithm.

Algorithm 6.5 CGLS

Given a least squares problem $\min \|\mathbf{G} \mathbf{m} - \mathbf{d}\|_2$, let $k = 0$, $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{p}_{-1} = \mathbf{0}$, $\beta_{-1} = 0$, $\mathbf{s}_0 = -\mathbf{d}$, and $\mathbf{r}_0 = \mathbf{G}^T \mathbf{s}_0$.

1. Let $\mathbf{p}_k = -\mathbf{r}_k + \beta_{k-1} \mathbf{p}_{k-1}$.
 2. Let $\alpha_k = \frac{\|\mathbf{r}_k\|_2^2}{(\mathbf{p}_k^T \mathbf{G}^T)(\mathbf{G} \mathbf{p}_k)}$.
 3. Let $\mathbf{m}_{k+1} = \mathbf{m}_k + \alpha_k \mathbf{p}_k$.
 4. Let $\mathbf{s}_{k+1} = \mathbf{s}_k + \alpha_k \mathbf{G} \mathbf{p}_k$.
 5. Let $\mathbf{r}_{k+1} = \mathbf{G}^T \mathbf{s}_{k+1}$.
 6. Let $\beta_k = \frac{\|\mathbf{r}_{k+1}\|_2^2}{\|\mathbf{r}_k\|_2^2}$.
 7. Let $k = k + 1$.
 8. Repeat the previous steps until convergence.
-

Note that this algorithm only requires one multiplication of $\mathbf{G}\mathbf{p}_k$ and one multiplication of $\mathbf{G}^T\mathbf{s}_{k+1}$ per iteration. We never explicitly compute $\mathbf{G}^T\mathbf{G}$, which might require considerable time, and which might have far more nonzero elements than $\mathbf{G}$ itself.

The CGLS algorithm has an important property that makes it particularly useful for ill-posed problems. It can be shown that, for exact arithmetic, $\|\mathbf{m}_k\|_2$ increases monotonically and $\|\mathbf{G}\mathbf{m}_k - \mathbf{d}\|_2$ decreases monotonically with increasing iterations [61, 65]. We can use the discrepancy principle together with this property to obtain a regularized solution. Simply stop the CGLS algorithm as soon as $\|\mathbf{G}\mathbf{m}_k - \mathbf{d}\|_2 < \delta$. In practice, this algorithm typically gives good solutions after a very small number of iterations.

An alternative way to use CGLS is to solve the Tikhonov regularization problem (4.4) by applying CGLS to

$$\min \left\| \begin{bmatrix} \mathbf{G} \\ \alpha \mathbf{L} \end{bmatrix} \mathbf{m} - \begin{bmatrix} \mathbf{d} \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (6.76)$$

For nonzero values of the regularization parameter α , this least squares problem should be reasonably well-conditioned. By solving this problem for several values of α , we can compute an L-curve. The disadvantage of this approach is that the number of CGLS iterations for each value of α may be large, and we need to solve the problem for several values of α . Thus the computational effort is far greater.

Example 6.3

A commonly used mathematical model of image blurring involves the two-dimensional convolution of the true image $I_{\text{true}}(x, y)$ with a **point spread function**, $\Psi(u, v)$ [14]:

$$I_{\text{blurred}}(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I_{\text{true}}(x - u, y - v) \Psi(u, v) \, du \, dv. \quad (6.77)$$

Here the point spread function shows how a point in the true image is altered in the blurred image. A point spread function that is commonly used to represent the blurring that occurs because an image is out of focus is the **Gaussian point spread function**

$$\Psi(u, v) = e^{-\frac{u^2 + v^2}{2\sigma^2}}. \quad (6.78)$$

Here the parameter σ controls the relative width of the point spread function. In practice, the blurred image and point spread function are discretized into pixels. In theory, Ψ is nonzero for all values of u and v . However, it becomes small quickly as u and v increase. If we set small values of Ψ to 0, then the $\mathbf{G}$ matrix in the discretized problem will be sparse.



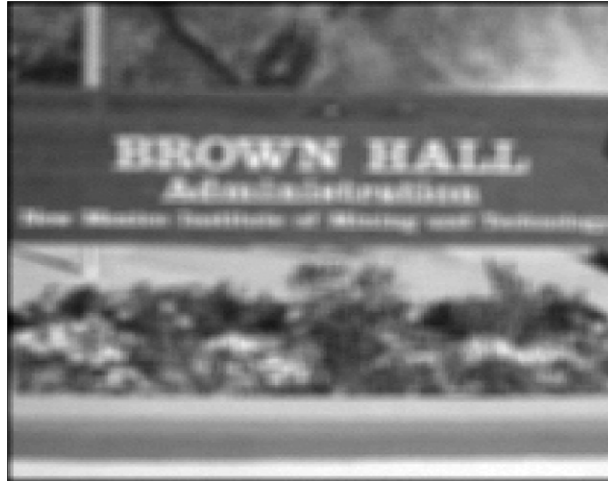


Figure 6.8 Blurred image.

Figure 6.8 shows an image that has been blurred and also has a small amount of added noise. This image is of size 200 pixels by 200 pixels, so the $\mathbf{G}$ matrix for the blurring operator is of size 40,000 by 40,000. Fortunately, the blurring matrix $\mathbf{G}$ is quite sparse, with less than 0.1% nonzero elements. The sparse matrix requires about 12 MB of storage. A dense matrix of this size would require about 13 GB of storage. Using the SVD approach to Tikhonov regularization would require far more storage than many current computers have. However, CGLS works quite well on this problem.

Figure 6.9 shows the L-curve for the solution of this problem by CGLS without explicit regularization and by CGLS with explicit regularization. The line with circles shows the solutions obtained by CGLS without explicit regularization. For the first 30 or so iterations of CGLS without explicit regularization, $\|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2$ decreases quickly. After that point, the improvement in misfit slows down, while $\|\mathbf{m}\|_2$ increases rapidly.

Figure 6.10 shows the CGLS solution without explicit regularization after 30 iterations. The blurring has been greatly improved. Note that 30 iterations is far less than the size of the matrix ($n = 40,000$). Unfortunately, further CGLS iterations do not significantly improve the image. In fact, noise builds up rapidly, both because of the accumulation of round-off errors and because the algorithm is converging slowly towards an unregularized least squares solution. Figure 6.11 shows the CGLS solution after 100 iterations. In this image the noise has been greatly amplified, with little or no improvement in the clarity of the image.

We also computed CGLS solutions with explicit Tikhonov regularization for 22 values of α . For each value of α , 200 iterations of CGLS were performed. The resulting

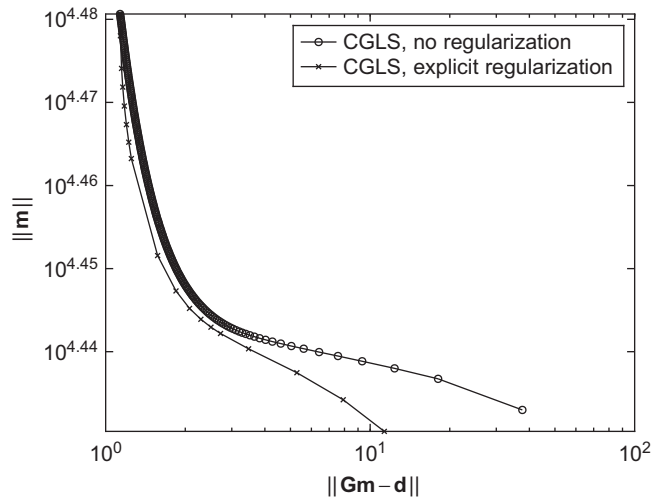


Figure 6.9 L-curves for CGLS deblurring.

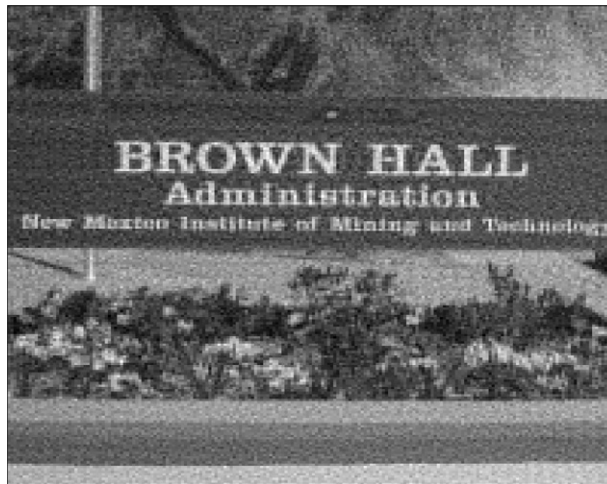


Figure 6.10 CGLS solution after 30 iterations, no explicit regularization.

L-curve is shown in [Figure 6.9](#) with “x” markers for each regularized solution that was obtained. This L-curve is slightly better than the L-curve from the CGLS solution without explicit regularization in that the values of $\|\mathbf{m}\|_2$ and $\|\mathbf{Gm} - \mathbf{d}\|_2$ are smaller. However, it required 40 times as much computational effort. The corner solution for $\alpha = 7.0 \times 10^{-4}$ is shown in [Figure 6.12](#). This solution is similar to the one shown in [Figure 6.10](#).

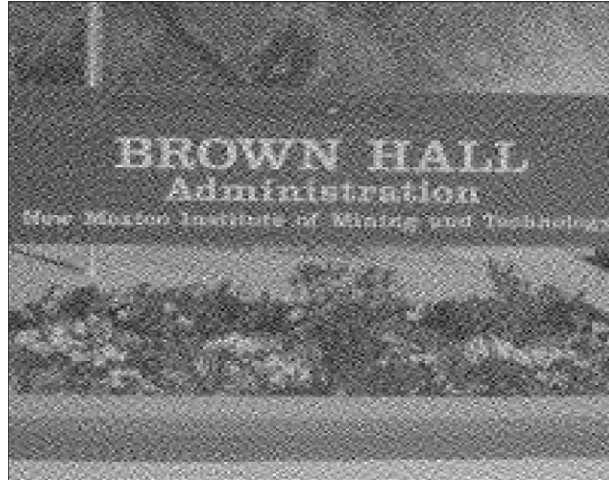


Figure 6.11 CGLS solution after 100 iterations, no explicit regularization.

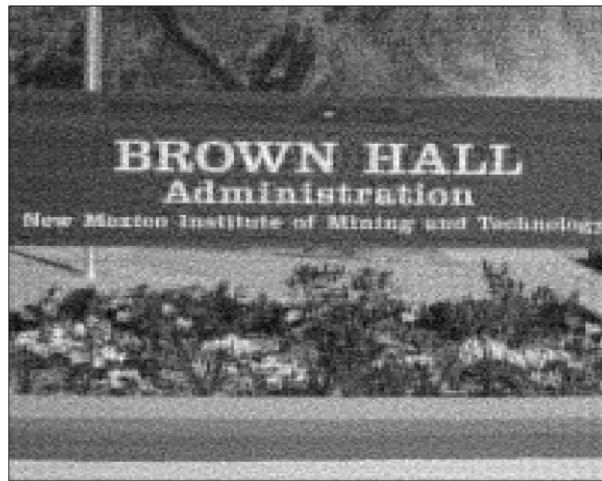


Figure 6.12 CGLS solution, explicit regularization, $\alpha = 7.0 \times 10^{-4}$.

We have focused on the CGLS method in this section because it is particularly easy to derive and because of the implicit regularization effect of the CGLS method. However, many other iterative methods have been developed for large-scale least squares problems [5, 135]. An analysis of the implicit regularizing effects of these methods can be found in Hanke [61]. The LSQR method of Paige and Saunders has been very widely used in many geophysical applications [123, 124]. MATLAB has an efficient and robust implementation of LSQR in its **lsqr** routine.

6.5. RESOLUTION ANALYSIS FOR ITERATIVE METHODS

Analyzing the resolution of a regularized solution computed by the iterative methods described in this chapter is a challenging problem. The formulas developed in Chapter 4 for the model resolution matrix under zeroth-order Tikhonov regularization were based on the SVD of the $\mathbf{G}$ matrix, but as we have seen it is often unfeasible to compute the SVD of a large matrix, and even when it is extremely sparse, the SVD is typically dense. For higher-order regularization, we have formulas for the resolution matrix based on the GSVD of the $\mathbf{G}$ matrix, such as (4.20), but it is also impractical to compute the GSVD of very large matrices.

A brute force approach that offers some advantages over the computation of the SVD or GSVD is to use (4.52) with Cholesky factorization of $(\mathbf{G}^T \mathbf{G} + \alpha^2 \mathbf{L}^T \mathbf{L})$ [19]. Cholesky factorization is considerably faster in practice than computing the SVD. However, since the Cholesky factor of this matrix is typically dense, this is still a very large-scale computation that would generally need to be performed in parallel on a computer with sufficient memory to store a dense n by n matrix.

If we want to compute an individual column of the model resolution matrix, this can be accomplished by solving an additional least squares problem. Applying the expression for the least squares Tikhonov inverse operator $\mathbf{G}^\sharp$ (4.52), the i th column of $\mathbf{R}_m$ can be expressed as

$$\begin{aligned} \mathbf{R}_{:,i} &= \mathbf{R}_m \mathbf{e}_i \\ &= \mathbf{G}^\sharp \mathbf{G} \mathbf{e}_i \\ &= \mathbf{G}^\sharp \mathbf{G}_{:,i} \\ &= (\mathbf{G}^T \mathbf{G} + \alpha^2 \mathbf{L}^T \mathbf{L})^{-1} \mathbf{G}^T \mathbf{G}_{:,i}. \end{aligned} \tag{6.79}$$

This is (4.26), a regularized least squares problem

$$\min \left\| \begin{bmatrix} \mathbf{G} \\ \alpha \mathbf{L} \end{bmatrix} \mathbf{R}_{:,i} - \begin{bmatrix} \mathbf{G}_{:,i} \\ \mathbf{0} \end{bmatrix} \right\|_2^2 \tag{6.80}$$

that can be solved for $\mathbf{R}_{:,i}$ using CGLS, LSQR, or some other iterative method. We could conceivably solve (6.80) to retrieve every column of the model resolution matrix in this manner, but this would require the solution of n least squares problems.

A number of authors have considered iterative techniques for computing low-rank approximations to the resolution matrix in combination with iterative least squares solution methods [12, 13, 174]. In such approaches, iterative methods are applied to compute the largest k singular values of a matrix and the associated singular vectors. For a sparse matrix, these methods are vastly more efficient than standard algorithms for computing the SVD.

Suppose that we obtain a low-rank (k singular values and vectors) approximation to the SVD

$$\mathbf{G} \approx \mathbf{U}_k \mathbf{L}_k \mathbf{V}_k^T \quad (6.81)$$

where $\mathbf{L}_k$ is a k by k nonsingular matrix, and $\mathbf{U}_k$ and $\mathbf{V}_k$ are matrices with k orthogonal columns. The pseudoinverse of $\mathbf{G}$ is then approximately

$$\mathbf{G}^\dagger \approx \mathbf{V}_k \mathbf{L}_k^{-1} \mathbf{U}_k^T. \quad (6.82)$$

Combining these approximations, we have

$$\mathbf{R}_m \approx \mathbf{V}_k \mathbf{V}_k^T. \quad (6.83)$$

Note that we would not want to explicitly multiply out $\mathbf{V}_k \mathbf{V}_k^T$, creating a huge and dense n by n resolution matrix. Rather, we could store the relatively small vector $\mathbf{V}_k$ and use it to compute desired entries of the $\mathbf{R}_m$ matrix as needed.

A more sophisticated version of this approach is to compute a low-rank approximation to the singular value decomposition of the augmented $\mathbf{G}$ matrix [173]. Consider an augmented system of equations

$$\mathbf{A} = \begin{bmatrix} \mathbf{G} \\ \alpha \mathbf{L} \end{bmatrix} \quad (6.84)$$

using truncated SVD approximations for $\mathbf{A}$ and its pseudoinverse, $\mathbf{A}^\dagger$,

$$\mathbf{A} \approx \mathbf{U}_k \mathbf{S}_k \mathbf{V}_k^T \quad (6.85)$$

and

$$\mathbf{A}^\dagger \approx \mathbf{V}_k \mathbf{S}_k^{-1} \mathbf{U}_k^T, \quad (6.86)$$

where $\mathbf{U}_k$ and $\mathbf{V}_k$ are composed of the first k columns of $\mathbf{U}$ and $\mathbf{V}$, and $\mathbf{S}_k$ is a square k by k matrix diagonal matrix of the largest k singular values.

It is convenient to write $\mathbf{U}_k$ as

$$\mathbf{U}_k = \begin{bmatrix} \mathbf{U}_G \\ \mathbf{U}_L \end{bmatrix}, \quad (6.87)$$

where $\mathbf{U}_G$ and $\mathbf{U}_L$ contain the rows of $\mathbf{U}_k$ corresponding to those of $\mathbf{G}$ and $\alpha \mathbf{L}$, respectively, in $\mathbf{A}$. We can thus partition (6.85) using

$$\mathbf{G} \approx \mathbf{U}_G \mathbf{S}_k \mathbf{V}_k^T. \quad (6.88)$$

Given (6.88) and (6.86), we can calculate an approximation of the model resolution matrix

$$\mathbf{R}_m = \mathbf{A}^\dagger \begin{bmatrix} \mathbf{G} \\ \mathbf{0} \end{bmatrix} \approx \mathbf{V}_k \mathbf{S}_k^{-1} \begin{bmatrix} \mathbf{U}_G^T & \mathbf{U}_L^T \end{bmatrix} \begin{bmatrix} \mathbf{U}_G \mathbf{S}_k \mathbf{V}_k^T \\ \mathbf{0} \end{bmatrix}, \quad (6.89)$$

which simplifies to

$$\mathbf{R}_m \approx \mathbf{V}_k \mathbf{S}_k^{-1} \mathbf{U}_G^T \mathbf{U}_G \mathbf{S}_k \mathbf{V}_k^T. \quad (6.90)$$

Using the identity

$$\mathbf{U}_G^T \mathbf{U}_G = \mathbf{I} - \mathbf{U}_L^T \mathbf{U}_L \quad (6.91)$$

we can alternatively write (6.90) as

$$\mathbf{R}_m \approx \mathbf{V}_k \left(\mathbf{I} - \mathbf{S}_k^{-1} \mathbf{U}_L^T \mathbf{U}_L \mathbf{S}_k \right) \mathbf{V}_k^T. \quad (6.92)$$

Depending on the relative sizes of $\mathbf{U}_G$ and $\mathbf{U}_L$, one or the other of (6.90) or (6.92) might be more easily evaluated. Again, we would probably not want to, or be able to, explicitly multiply out the matrices to obtain the entire dense n by n model resolution matrix for a large problem, but could instead use (6.90) or (6.92) to compute specific desired elements of the resolution matrix.

In practice, it may not be feasible for very large problems to compute, or even store, a sufficient number of singular values and vectors to adequately approximate the model resolution matrix using low-rank, SVD-based approximations [37]. We will thus consider an alternative stochastic approach based on [9] that estimates solely the resolution matrix diagonal, but which is not limited by low-rank matrix approximations.

Consider a sequence of s random vectors from $\mathbf{R}^n$, $\mathbf{v}_1, \dots, \mathbf{v}_s$, with independent elements drawn from a standard normal distribution. The s th corresponding estimate for the diagonal of an n by n square matrix $\mathbf{A}$ is given by

$$\mathbf{q}_s = \left[\sum_{k=1}^s \mathbf{v}_k \odot \mathbf{A} \mathbf{v}_k \right] \oslash \left[\sum_{k=1}^s \mathbf{v}_k \odot \mathbf{v}_k \right] \quad (6.93)$$

where $\odot$ denotes element-wise multiplication and $\oslash$ denotes element-wise division of vectors. Note that the only way in which $\mathbf{A}$ is used within this algorithm is in matrix-vector multiplications, so as long as we have a function that can implement the matrix vector multiplication, then we can implement the algorithm. As we have already seen, the multiplication of a large matrix with a vector can be effected by solving a least squares problem (6.80). Specifically, to solve for the k th product $\mathbf{y} = \mathbf{R}_m$, we solve [103]

$$\min \left\| \begin{bmatrix} \mathbf{G} \\ \alpha \mathbf{L} \end{bmatrix} \mathbf{y} - \begin{bmatrix} \mathbf{G} \mathbf{v}_k \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \quad (6.94)$$

It is necessary to solve s least squares systems of equations (6.94) to calculate (6.93). In practice, relatively small values of s in (6.93) (e.g., 100 to 1000) are adequate even when there are tens or hundreds of thousands of model parameters [103].

The stochastic algorithm (6.93) can also be used to implement generalized cross-validation. In (4.74), the most difficult computation is evaluation of the trace of

$\mathbf{I} - \mathbf{G}\mathbf{G}^\sharp$. Computing this matrix explicitly and then evaluating the trace would require excessive time and storage. Using (6.93) and evaluating the necessary $\mathbf{G}^\sharp$ matrix-vector multiplications by solving an associated least squares problem using an iterative method such as LSQR or CGLS makes it tractable to compute an approximation to the GCV function (4.74), where we solve s least squares problems for each value of α [103].

Example 6.4

We will examine the solution to a large-scale tomography problem involving 30,000 rays passing through a cubic volume discretized into a 30 by 30 by 30 model of 27,000 cells (Figure 6.13).

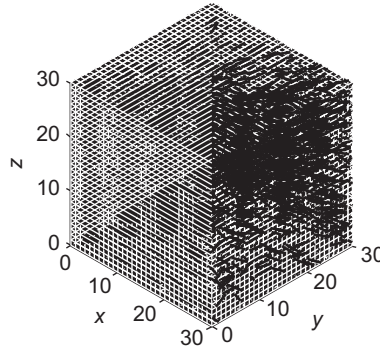


Figure 6.13 A 30 by 30 by 30 block tomography model interrogated by 30,000 ray paths.

Ray path start and end points are uniformly randomly distributed across the sides of the volume with 10,000 rays passing between each pair of parallel sides. The corresponding $\mathbf{G}$ matrix is 30,000 by 27,000, but only 0.2% of its entries are nonzero. Synthetic travel time data (with travel time perturbation values between approximately -20 and 30 time units) were generated from a zero-mean smooth model with $N(0, (0.5)^2)$ noise added. We implemented second-order Tikhonov regularization with a small amount of additional zeroth-order regularization to regularize the solution.

Figure 6.14 shows the L-curve for this problem. Figure 6.15 shows a plot of the GCV function (4.74) computed using stochastic estimates of the diagonal of $\mathbf{I} - \mathbf{G}\mathbf{G}^\sharp$. The L-curve corner and GCV minimum both occur at about $\alpha = 0.5$. For the solution obtained using $\alpha = 0.5$, we computed a stochastic estimate of the diagonal of the resolution matrix using (6.93). For comparison, we also computed exact resolution matrix diagonal element values for 100 randomly selected indices using (6.79). Figure 6.16 shows a scatter plot of the stochastic estimates compared with corresponding exact determinations, and indicates that the stochastic estimates are accurate to a few percentage

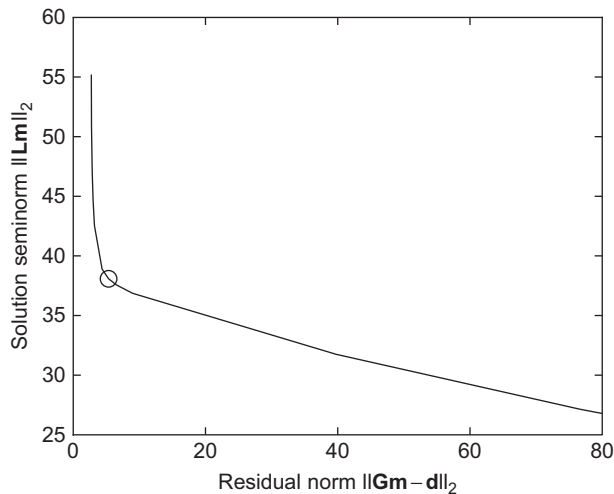


Figure 6.14 L-curve for the tomography example of Figure 6.13 and its corner near $\alpha = 0.5$.

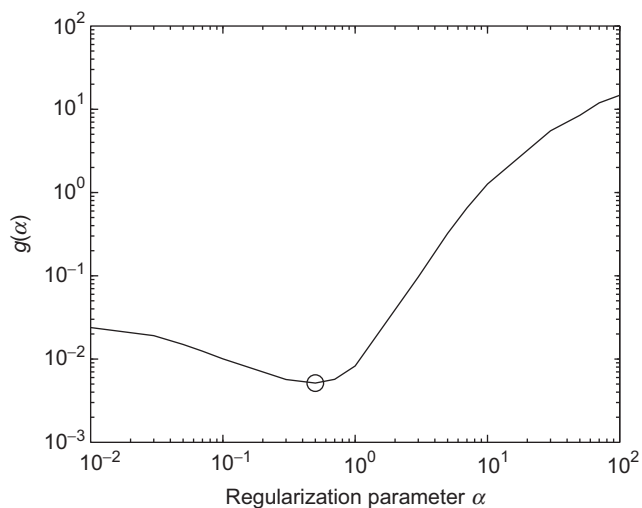


Figure 6.15 GCV plot for the tomography example of Figure 6.13 and its minimum near $\alpha = 0.5$.

points. It is common to use counts of the number of rays transecting each cell as a proxy for assessing the diagonal of the resolution matrix in tomographic problems. In general, the association between ray density and resolution in a tomography problem will depend on the specific ray path geometry. Figure 6.17 shows a scatter plot of the ray densities versus the accurately computed values of the diagonal elements of $\mathbf{R}_m$ for the 100 random indices plotted in Figure 6.16. The relationship shows a nonlinear trend between ray density and the resolution diagonal element values, and there are also anomalously

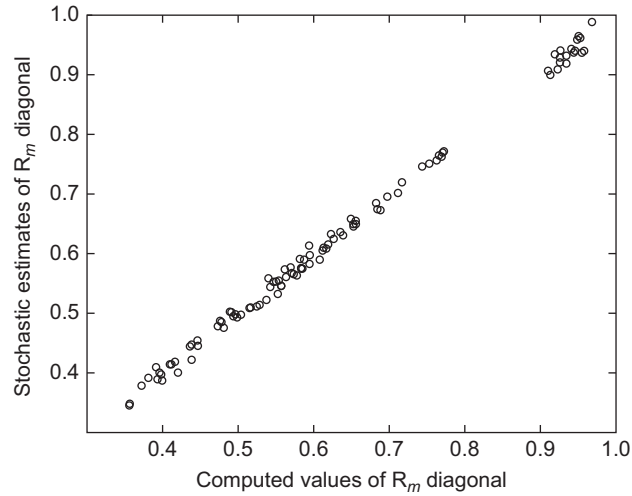


Figure 6.16 Stochastic estimates versus computed values of the diagonal of R_m .

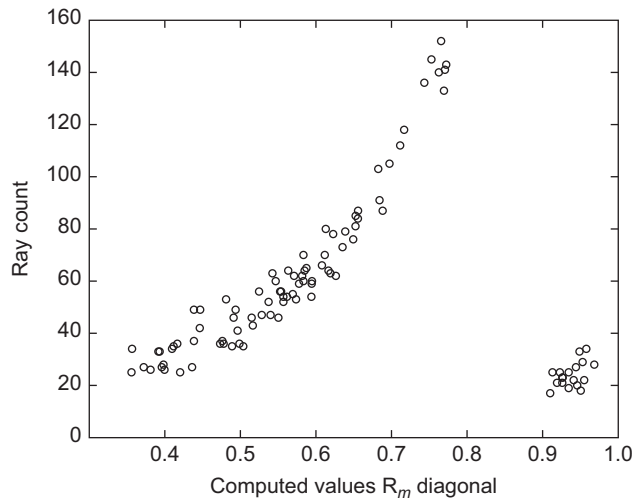


Figure 6.17 Comparison of ray densities and resolution.

well-resolved outliers in the plot with resolution diagonal element values greater than 0.9. Further examination shows that the anomalously well-resolved outliers correspond to cells that reside on the model exterior and are thus constrained by two-dimensional ray path geometries.

6.6. EXERCISES

1. Consider the cross-well tomography problem of Exercise 4.3.
 - a. Apply Kaczmarz's algorithm to this problem.
 - b. Apply ART to this problem.
 - c. Apply SIRT to this problem.
 - d. Comment on the solutions that you obtained.
2. A very simple iterative regularization method is the **Landweber iteration** [67]. The algorithm begins with $\mathbf{m}^{(0)} = \mathbf{0}$, and then follows the iteration,

$$\mathbf{m}^{(k+1)} = \mathbf{m}^{(k)} - \omega(\mathbf{G}\mathbf{m}^{(k)} - \mathbf{d}). \quad (6.95)$$

To ensure convergence, the parameter ω must be selected so that $0 < \omega < 2/s_1^2$, where s_1 is the largest singular value of $\mathbf{G}$.

In practice, the CGLS method generally works better than the Landweber iteration. However, it is easier to analyze the performance of the Landweber iteration. It can be shown that the k th iterate of the Landweber iteration is exactly the same as the SVD solution with filter factors of

$$f_i^{(k)} = 1 - (1 - \omega s_i^2)^k. \quad (6.96)$$

- a. Implement the Landweber iteration and apply it to the Shaw problem from Example 4.3.
- b. Verify that $\mathbf{m}^{(10)}$ from the Landweber iteration matches the SVD solution with filter factors given by (6.96).
- c. Derive (6.96).
3. The Landweber iteration described in the previous exercise converges under the condition that $0 < \omega < 2/s_1^2$, where s_1 is the largest singular value of $\mathbf{G}$ (or equivalently, $s_1 = \|\mathbf{G}\|_2$). As a practical matter we typically cannot compute the full SVD of $\mathbf{G}$. However, it is possible to rapidly estimate this quantity using an iterative method that we will derive in this exercise. Recall from Appendix A that $\|\mathbf{G}\|_2 = s_1$ is the square root of the largest eigenvalue of the matrix $\mathbf{G}^T\mathbf{G}$.
 - a. Using (A.77), diagonalize the matrix $\mathbf{A} = \mathbf{G}^T\mathbf{G}$ and use the diagonalization to show that

$$\mathbf{A}^n = \mathbf{P}\mathbf{\Lambda}^n\mathbf{P}^{-1}. \quad (6.97)$$

Assume that the eigenvalues of $\mathbf{A}$ are sorted so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$.

- b. Take an arbitrary vector $\mathbf{x}$ in $\mathbf{R}^n$, and write it in terms of the eigenvectors of $\mathbf{A}$ as

$$\mathbf{x} = \alpha_1 \mathbf{v}_1 + \cdots + \alpha_n \mathbf{v}_n. \quad (6.98)$$

Then show that

$$\mathbf{A}^k \mathbf{x} = \alpha_1 \lambda_1^k \mathbf{v}_1 + \cdots + \alpha_n \lambda_n^k \mathbf{v}_n. \quad (6.99)$$

- c. Starting with a random vector $\mathbf{x}_0$, and assuming that $\alpha_1 \neq 0$ (a reasonable assumption for a random vector $\mathbf{x}_0$), show that

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{A}^k \mathbf{x}_0\|_2}{\|\mathbf{A}^{k-1} \mathbf{x}_0\|_2} = \lambda_1. \quad (6.100)$$

This leads to a simple iterative algorithm for estimating $s_1 = \sqrt{\lambda_1}$. Start with a random vector $\mathbf{x}_0$. In each iteration, let

$$\mathbf{x}_{k+1} = \frac{\mathbf{G}^T(\mathbf{G}\mathbf{x}_k)}{\|\mathbf{x}_k\|_2} \quad (6.101)$$

and let

$$\rho_k = \sqrt{\|\mathbf{G}^T(\mathbf{G}\mathbf{x}_k)\|_2}. \quad (6.102)$$

The sequence ρ_k converges to s_1 . This function is implemented in MATLAB as the **normest** function. Write your own implementation of this function and compare the results you obtain with **normest**.

4. Use the Landweber iteration discussed in Exercise 6.2 to deblur the image from [Example 6.3](#). In order to use the Landweber iteration, you will need to estimate $s_1 = \|\mathbf{G}\|_2$, which can be done using the **normest** command discussed in Exercise 6.3.
5. In [Example 6.3](#) we have included a function **blur** that computes the system matrix for the problem of deblurring an image that has been blurred by a Gaussian point spread function. The file **blur.mat** contains a blur-inducing sparse $\mathbf{G}$ matrix and a data vector $\mathbf{d}$, where the 100 by 100 pixel gray-scale image is mapped into $\mathbf{d}$ using a column-by-column indexing convention.
 - a. How large is the $\mathbf{G}$ matrix? How many nonzero elements does it have? How much storage would be required for the $\mathbf{G}$ matrix if all of its elements were nonzero? How much storage would the SVD of $\mathbf{G}$ require?
 - b. Plot the raw image.
 - c. Using the MATLAB **cgls** algorithm with 100 iterations, deblur the image by solving $\mathbf{G}\mathbf{m} = \mathbf{d}$ and interpret the evolution of your solutions with increasing iterations.
6. Show that if $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{n-1}$ are nonzero and mutually conjugate with respect to an $n \times n$ symmetric and positive definite matrix $\mathbf{A}$, then the vectors are also linearly independent. Hint: Use the definition of linear independence.
7. Recall the iteratively reweighted least squares method introduced in Chapter 2. The system of equations (2.92) solved in each iteration of the IRLS algorithm might well



be dense, even if the matrix $\mathbf{G}$ is sparse. However, this system of equations can also be thought of as the normal equations for a particular least squares problem that will be sparse when $\mathbf{G}$ is sparse. Write down this least squares problem. Write a MATLAB function that uses the **lsqr** routine to solve these least squares problems in an IRLS scheme to minimize $\|\mathbf{G}\mathbf{m} - \mathbf{d}\|_1$. Hint: Use the fact that $\mathbf{R}$ is a diagonal matrix with positive elements; you can take its square root.

6.7. NOTES AND FURTHER READING

Iterative methods for the solution of linear systems of equations are an important topic in numerical analysis. Some basic references include [5, 87, 135, 142].

Iterative methods for tomography problems including Kaczmarz's algorithm, ART, and SIRT are discussed in [84, 115, 163], and parallel algorithms based on these methodologies are discussed in [30]. These methods are often referred to as **row action methods** because they access only one row of the matrix at a time, which makes them relatively easy to implement in parallel processing. In practice, the conjugate gradient and LSQR methods generally provide better performance than the row action methods. There are some interesting connections between SIRT and the conjugate gradient method discussed in [120, 146, 147].

Hestenes and Stiefel are generally credited with the invention of the conjugate gradient method [72]. However, credit is also due to Lanczos [92]. The history of the conjugate gradient method is discussed in [55, 71]. Shewchuk's technical report [142] provides an introduction to the conjugate gradient method with illustrations that help to make the geometry of the method very clear. Filter factors for the CGLS method similar to those in Exercise 2 can be determined. These are derived in [65]. The LSQR method of Paige and Saunders [65, 123, 124] is an alternative way to apply the CG method to the normal equations. The resolution of LSQR solutions is discussed in [12, 13]. Schemes have been developed for using CGLS with explicit regularization and dynamic adjustment of the regularization parameter α [85, 86, 106]. This can potentially remove the computational burden of solving the problem for many values of α . An alternative approach can be used to compute regularized solutions for several values of α at once [50]. The performance of the CG algorithm degrades dramatically on poorly conditioned systems of equations. In such situations a technique called **preconditioning** can be used to improve the performance of CG. Essentially, preconditioning involves a change of variables $\bar{\mathbf{x}} = \mathbf{C}\mathbf{x}$. The matrix $\mathbf{C}$ is selected so that the resulting system of equations will be better conditioned than the original system of equations [38, 53, 164]. The conjugate gradient method can also be generalized to nonlinear minimization problems [58, 142].

Inverse problems in image processing are a very active area of research. Some books on inverse problems in imaging include [14, 115].